

UNIVERSIDAD DEL VALLE
Facultad de Ingeniería
Escuela de Ingeniería de Sistemas y Computación
Análisis de Algoritmos II

Informe de Proyecto

Proyecto de curso

Calculando el plan de riego óptimo de una finca

Estudiante:	Juan Carlos Cruz Muñoz (1824389)
Integrantes:	David Alejandro Enciso, Juan Esteban, Estiven
Profesores:	Juan Francisco Díaz Frías, Jesús Alexander Aranda
Monitor:	Samuel Galindo

Cali, Colombia
Mayo de 2026

Índice

1. Introducción	2
2. El Problema del Riego Óptimo	2
2.1. Descripción del Problema	2
2.2. Formalización Matemática	3
3. Solución Mediante Fuerza Bruta	4
3.1. Complejidad	4
3.2. Corrección	4
4. Solución Mediante Algoritmo Voraz	4
4.1. Descripción de la Estrategia	4
4.1.1. Codificación e Implementación	6
4.2. Análisis de Resultados	8
4.3. Complejidad y Corrección	14
5. Solución Mediante Programación Dinámica	17
5.1. Subestructura Óptima	17
5.2. Definición Recursiva	17
5.3. Análisis de Complejidad Temporal y Espacial	18
6. Comparación de Estrategias	18
7. Conclusiones	19

1. Introducción

Este informe desarrolla el problema del riego óptimo planteado en el enunciado del proyecto de curso. El objetivo es construir y analizar tres enfoques para programar el orden de riego de los tablones de una finca: fuerza bruta, algoritmo voraz y programación dinámica. Cada estrategia se estudia desde tres perspectivas: su idea algorítmica, su complejidad y su capacidad para producir una solución óptima.

La implementación del repositorio fue realizada en C++ y concentra la lógica principal en el archivo `backend/src/algoritmos.cpp`. En ese archivo se encuentran las funciones `roFB`, `roV` y `roPD`, que corresponden respectivamente a los enfoques de fuerza bruta, voraz y programación dinámica solicitados por el enunciado. Además, el proyecto incluye una interfaz para cargar archivos de entrada, ejecutar cualquiera de los algoritmos y revisar las salidas obtenidas.

La dificultad principal del problema está en que el costo no solo depende de qué tablón se riega, sino también del instante exacto en que comienza su riego. Esto hace que el orden completo de la programación sea determinante. Como consecuencia, una decisión aparentemente buena en el corto plazo puede afectar negativamente el costo global. Por eso resulta pertinente comparar una estrategia exhaustiva, una heurística voraz y una formulación exacta mediante programación dinámica.

2. El Problema del Riego Óptimo

2.1. Descripción del Problema

Una finca está compuesta por varios tablones y se dispone de un único recurso móvil de riego. Cada tablón tiene cuatro atributos relevantes:

- **Tiempo de supervivencia** (ts_i): número máximo de días que puede pasar sin ser regado antes de sufrir afectación grave.
- **Tiempo de riego** (tr_i): días requeridos para regar completamente el tablón.
- **Prioridad** (p_i): nivel de importancia del tablón, entre 1 y 4.
- **Tiempo de riego perfecto** (rp_i): instante ideal para iniciar el riego del tablón.

Como solo existe un recurso de riego, en cada instante se debe decidir qué tablón atender primero. El problema consiste en hallar un orden de riego que minimice el costo total de afectación de los cultivos. La función de costo del enunciado penaliza tres situaciones:

- si el tablón se riega exactamente en su día perfecto, se obtiene el costo más bajo;
- si se riega antes de superar su límite de supervivencia, pero no en el instante perfecto, el costo es moderado;
- si se riega tarde, la penalización aumenta y además se multiplica por la prioridad del tablón.

En términos prácticos, el problema es una variante de secuenciamiento con una función objetivo dependiente del tiempo de inicio de cada tarea. No basta con identificar tablones “urgentes”; también es importante reconocer que a veces conviene postergar un riego para acercarse al día perfecto o al menos no adelantar innecesariamente un tablón cuyo costo disminuye si se atiende más tarde.

2.2. Formalización Matemática

Sea una finca

$$F = \langle T_0, T_1, \dots, T_{n-1} \rangle,$$

donde cada tablón se define como

$$T_i = \langle ts_i, tr_i, p_i, rp_i \rangle,$$

con $0 \leq rp_i \leq ts_i - tr_i$.

Una programación de riego es una permutación

$$\Pi = \langle \pi_0, \pi_1, \dots, \pi_{n-1} \rangle$$

de los índices $\{0, 1, \dots, n-1\}$, que indica el orden en el que se riegan los tablones.

El instante de inicio del riego de cada tablón depende de los tablones programados antes que él. Si t_i^Π representa el instante en que empieza a regarse el tablón i bajo la programación Π , entonces:

$$t_{\pi_0}^\Pi = 0, \quad t_{\pi_j}^\Pi = \sum_{k=0}^{j-1} tr_{\pi_k} \quad \text{para } j \geq 1.$$

La contribución al costo del tablón i al empezar a regarse en el instante t_i^Π es:

$$CR_F^\Pi[i] = \begin{cases} ts_i - (t_i^\Pi + tr_i), & \text{si } t_i^\Pi = rp_i, \\ 2(ts_i - (t_i^\Pi + tr_i)), & \text{si } t_i^\Pi \neq rp_i \text{ y } t_i^\Pi \leq ts_i - tr_i, \\ 2p_i((t_i^\Pi + tr_i) - ts_i), & \text{en otro caso.} \end{cases}$$

El costo total de la programación es:

$$CR_F^\Pi = \sum_{i=0}^{n-1} CR_F^\Pi[i].$$

El problema del riego óptimo consiste entonces en encontrar una permutación Π^* tal que:

$$\Pi^* = \arg \min_{\Pi} CR_F^\Pi.$$

3. Solución Mediante Fuerza Bruta

La estrategia de fuerza bruta considera todas las posibles permutaciones de los n tablones. Para cada orden se simula el avance del tiempo, se calcula el costo de cada tablón con la función anterior y finalmente se escoge la permutación de menor costo total.

En la implementación del proyecto, la función `roFB` genera las permutaciones usando la rutina estándar `next_permutation`. Se conserva el mejor orden visto hasta el momento y, al finalizar el recorrido exhaustivo, se vuelve a evaluar ese orden para construir la salida completa con detalles por tablón.

3.1. Complejidad

El número de permutaciones posibles de n tablones es $n!$. Evaluar una permutación requiere recorrer los n tablones una vez, por lo que el costo de evaluación es $O(n)$. En consecuencia, el tiempo total es:

$$O(n! \cdot n).$$

El espacio auxiliar es $O(n)$, ya que solo se almacenan la permutación actual, la mejor permutación encontrada y algunas variables escalares. Sin embargo, aunque el consumo de memoria es moderado, el crecimiento factorial del tiempo vuelve a esta solución inviable para instancias medianas. En la práctica, resulta razonable solo para valores pequeños de n .

3.2. Corrección

La corrección de la fuerza bruta es directa. El conjunto de soluciones factibles del problema es precisamente el conjunto de todas las permutaciones de los n tablones. Como el algoritmo recorre todas esas permutaciones y calcula exactamente su costo, necesariamente inspecciona también la solución óptima global. Al escoger al final la permutación de menor costo entre todas las evaluadas, el resultado devuelto coincide con la definición del óptimo del problema.

Por tanto, la estrategia de fuerza bruta siempre produce la respuesta correcta.

4. Solución Mediante Algoritmo Voraz

4.1. Descripción de la Estrategia

La implementación voraz del proyecto corresponde a la función `roV`. El algoritmo recibe una finca

$$F = \langle T_0, \dots, T_{n-1} \rangle$$

y construye una permutación Π_V que define el orden de riego. Para cada tablón i calcula el valor

$$\rho_i = \frac{tr_i}{p_i},$$

y luego ordena los tablonos de menor a mayor según ρ_i . Si dos tablonos empatan en ese valor, se desempata usando el margen

$$ts_i - tr_i,$$

privilegiando primero el tablón con menor margen.

Intuición. Si dos tablonos ya están en el caso tardío, el costo de regar i en el instante t queda dominado por

$$2p_i((t + tr_i) - ts_i).$$

Aquí aparecen dos fuerzas: un tr_i grande retrasa más a los demás, y un p_i grande hace más caro ese retraso. Por eso el cociente $\frac{tr_i}{p_i}$ intenta equilibrar duración y penalización.

Comparación local. Supongamos ahora que dos tablonos i y j ya están en caso 3 en el instante t . Si se riegan en el orden $i \rightarrow j$, el costo conjunto es

$$C(i \rightarrow j) = 2p_i(t + tr_i - ts_i) + 2p_j(t + tr_i + tr_j - ts_j).$$

Si se riegan en el orden $j \rightarrow i$, el costo conjunto es

$$C(j \rightarrow i) = 2p_j(t + tr_j - ts_j) + 2p_i(t + tr_j + tr_i - ts_i).$$

Al restar ambas expresiones se obtiene

$$\Delta_{ij} = C(i \rightarrow j) - C(j \rightarrow i) = 2(p_j tr_i - p_i tr_j).$$

Como Δ_{ij} mide cuánto más caro es elegir el orden $i \rightarrow j$ en vez de $j \rightarrow i$, ocurren tres casos:

- si $\Delta_{ij} < 0$, el orden $i \rightarrow j$ cuesta menos;
- si $\Delta_{ij} = 0$, ambos órdenes cuestan lo mismo;
- si $\Delta_{ij} > 0$, el orden $j \rightarrow i$ cuesta menos.

Por tanto, conviene poner i antes que j exactamente cuando $\Delta_{ij} \leq 0$, es decir, cuando

$$\frac{tr_i}{p_i} \leq \frac{tr_j}{p_j}.$$

Conclusión. En el subproblema donde ambos tablonos ya están en penalización tardía, el mejor orden local se obtiene comparando $\frac{tr}{p}$. Esa es la base formal del criterio principal del algoritmo. Si dos tablonos empatan en ese cociente, se adelanta el de menor holgura $ts_i - tr_i$, porque tiene menos margen para seguir esperando. Esto no prueba optimalidad global, pero sí hace más coherente el comportamiento del voraz con la estructura del problema.

Desde el punto de vista computacional, la estrategia tiene tres pasos:

Paso	Operación	Complejidad
1	Calcular $\frac{tr_i}{p_i}$ y $ts_i - tr_i$ para cada tablón	$O(n)$
2	Ordenar por $\frac{tr}{p}$ y desempatar por $ts - tr$	$O(n \log n)$
3	Simular la permutación y calcular los costos	$O(n)$

Tabla 1: Desglose operacional del algoritmo voraz.

4.1.1. Codificación e Implementación

La implementación de **roV** no reordena directamente los tableros, sino sus índices. Para simplificar la escritura, definamos

$$\rho_i = \frac{tr_i}{p_i}, \quad h_i = ts_i - tr_i.$$

Entonces, para dos tableros i y j , el comparador usado por **sort** aplica estas reglas en orden:

1. Si $\rho_i < \rho_j$, entonces i va antes que j .
2. Si $\rho_i = \rho_j$, va primero el tablón con menor holgura, es decir, el que tenga menor h .
3. Si también empatan en holgura, va primero el de menor índice original.

Es decir, el algoritmo prioriza primero la razón $\frac{tr}{p}$, luego la holgura $ts - tr$, y finalmente el índice para romper empates de manera determinista.

Como el código evita divisiones en punto flotante, la comparación principal no se implementa calculando cocientes decimales, sino mediante producto cruzado:

$$\frac{tr_i}{p_i} < \frac{tr_j}{p_j} \iff tr_i p_j < tr_j p_i.$$

Proposición. El paso de ordenamiento del algoritmo voraz cuesta $\Theta(n \log n)$.

Demostración. Sea n el número de tableros.

$$\text{iota} = O(n).$$

Además, cada invocación del comparador realiza un número constante de operaciones aritméticas y comparaciones enteras; por tanto, cada comparación cuesta $O(1)$. Como la biblioteca estándar garantiza que **std::sort**¹ realiza $O(n \log n)$ comparaciones, se obtiene la cota superior

$$T_{\text{sort}}(n) = O(n \log n).$$

Para la cota inferior, basta observar que ordenar n elementos distintos exige distinguir entre las $n!$ permutaciones posibles. Cada comparación solo tiene dos resultados, así que

¹Véase el borrador del estándar C++, sección [\[sort\]](#), donde se especifica la complejidad de **std::sort** como $O(N \log N)$ comparaciones y proyecciones.

después de h comparaciones solo pueden diferenciarse a lo sumo 2^h casos. Por lo tanto, necesariamente debe cumplirse

$$2^h \geq n!,$$

y al tomar logaritmos se obtiene

$$h \geq \log_2(n!).$$

Ahora bien, en el producto $n! = 1 \cdot 2 \cdot \dots \cdot n$, los últimos $n/2$ factores son al menos $n/2$. Entonces

$$n! \geq \left(\frac{n}{2}\right)^{n/2},$$

y por tanto

$$h \geq \log_2(n!) \geq \frac{n}{2} \log_2\left(\frac{n}{2}\right) = \Omega(n \log n).$$

En consecuencia, cualquier ordenamiento por comparaciones requiere $\Omega(n \log n)$ comparaciones en el peor caso. Como ya se tenía la cota superior $O(n \log n)$, se concluye que

$$T_{\text{sort}}(n) = \Theta(n \log n).$$

Finalmente, como `evaluar_orden` recorre la permutación una sola vez, su costo es $O(n)$ y no modifica el orden asintótico total de `roV`.

El siguiente fragmento muestra el núcleo de esa implementación:

```
vector<int> orden(cantidad_tablones);
iota(orden.begin(), orden.end(), 0);

sort(orden.begin(), orden.end(), [&](int indice_a, int indice_b) {
    const Tablon& a = tablones[indice_a];
    const Tablon& b = tablones[indice_b];

    long long ratio_a = 1LL * a.tiempo_riego * b.prioridad;
    long long ratio_b = 1LL * b.tiempo_riego * a.prioridad;
    if (ratio_a != ratio_b) return ratio_a < ratio_b;

    const int margen_a = a.tiempo_supervivencia - a.tiempo_riego;
    const int margen_b = b.tiempo_supervivencia - b.tiempo_riego;
    if (margen_a != margen_b) return margen_a < margen_b;

    return indice_a < indice_b;
});
```

Brevemente, cada bloque del fragmento cumple una función específica:

- `vector<int>orden(cantidad_tablones);` reserva el arreglo donde se guardará la permutación de índices que luego se ordenará.
- `iota(orden.begin(), orden.end(), 0);` inicializa ese arreglo como $\langle 0, 1, \dots, n-1 \rangle$, es decir, con el orden original de los tablones.
- `sort(...)` aplica el comparador voraz sobre esos índices para construir la permutación final.

- `const Tablon& a = tablon[indice_a];` recupera el primer tablón involucrado en la comparación.
- `const Tablon& b = tablon[indice_b];` recupera el segundo tablón involucrado en la comparación.
- `ratio_a` y `ratio_b` implementan el producto cruzado $tr_i p_j$ y $tr_j p_i$, evitando dividir en punto flotante.
- `if (ratio_a != ratio_b) return ratio_a < ratio_b;` decide el orden principal según la regla $\frac{tr_i}{p_i} < \frac{tr_j}{p_j}$.
- `margen_a` y `margen_b` calculan la holgura $ts - tr$ de cada tablón para usarla como segundo criterio.
- `if (margen_a != margen_b) return margen_a < margen_b;` da prioridad al tablón con menor margen cuando la razón principal empata.
- `return indice_a < indice_b;` resuelve el empate final conservando el orden por índice y hace determinista la salida del algoritmo.

Ejemplo. Si T_i tiene $(tr_i, p_i) = (2, 4)$ y T_j tiene $(tr_j, p_j) = (3, 2)$, el código no compara $2/4$ con $3/2$, sino

$$2 \cdot 2 = 4 \quad \text{y} \quad 3 \cdot 4 = 12.$$

Como $4 < 12$, decide $i \prec j$. Si dos tablonas empatan en $\frac{tr}{p}$, por ejemplo $(ts_i, tr_i, p_i) = (6, 2, 2)$ y $(ts_j, tr_j, p_j) = (9, 3, 3)$, entonces ambos tienen razón 1 y el desempate pasa a la holgura:

$$ts_i - tr_i = 4 \quad < \quad ts_j - tr_j = 6.$$

Por eso el algoritmo coloca primero a i .

4.2. Análisis de Resultados

Aplicación a los ejemplos del enunciado. Para las dos fincas de la Sección 2.3 del enunciado, el criterio voraz produce la misma permutación:

$$\Pi_V = \langle 0, 1, 3, 2, 4 \rangle.$$

Ejemplo 1. Para

$$F_1 = \langle \langle 10, 3, 4, 0 \rangle, \langle 6, 3, 3, 1 \rangle, \langle 2, 2, 1, 0 \rangle, \langle 8, 1, 1, 6 \rangle, \langle 10, 4, 2, 5 \rangle \rangle$$

los valores relevantes son:

El orden voraz es

$$T_0 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2 \rightarrow T_4.$$

Al simular esa programación se obtiene:

$$CR_{F_1}^{\Pi_V} = 7 + 0 + 1 + 14 + 12 = 34.$$

Tablón	ts	tr	p	rp	$\frac{tr}{p}$	$ts - tr$
T_0	10	3	4	0	0.75	7
T_1	6	3	3	1	1.00	3
T_2	2	2	1	0	2.00	0
T_3	8	1	1	6	1.00	7
T_4	10	4	2	5	2.00	6

Tabla 2: Valores usados por el voraz en la finca F_1 .

Paso	Tablón	t_{inicio}	Caso aplicado	Costo
1	T_0	0	caso 1: $10 - (0 + 3)$	7
2	T_1	3	caso 2: $2(6 - (3 + 3))$	0
3	T_3	6	caso 1: $8 - (6 + 1)$	1
4	T_2	7	caso 3: $2 \cdot 1 \cdot ((7 + 2) - 2)$	14
5	T_4	9	caso 3: $2 \cdot 2 \cdot ((9 + 4) - 10)$	12

Tabla 3: Evaluación detallada del voraz sobre F_1 .

La programación óptima por fuerza bruta es

$$\Pi^* = \langle 2, 1, 0, 3, 4 \rangle$$

con costo 20. La diferencia se explica porque el voraz posterga T_2 , que tiene holgura cero ($ts_2 - tr_2 = 0$), y lo condena a caer en caso 3.

Ejemplo 2. Para

$$F_2 = \langle \langle 9, 3, 4, 0 \rangle, \langle 5, 3, 3, 2 \rangle, \langle 2, 2, 1, 0 \rangle, \langle 8, 1, 1, 6 \rangle, \langle 6, 4, 2, 1 \rangle \rangle$$

se obtiene la misma permutación voraz. La evaluación queda:

Paso	Tablón	t_{inicio}	Caso aplicado	Costo
1	T_0	0	caso 1: $9 - (0 + 3)$	6
2	T_1	3	caso 3: $2 \cdot 3 \cdot ((3 + 3) - 5)$	6
3	T_3	6	caso 1: $8 - (6 + 1)$	1
4	T_2	7	caso 3: $2 \cdot 1 \cdot ((7 + 2) - 2)$	14
5	T_4	9	caso 3: $2 \cdot 2 \cdot ((9 + 4) - 6)$	28

Tabla 4: Evaluación detallada del voraz sobre F_2 .

$$CR_{F_2}^{\Pi_V} = 6 + 6 + 1 + 14 + 28 = 55.$$

El óptimo es nuevamente

$$\Pi^* = \langle 2, 1, 0, 3, 4 \rangle$$

con costo 32. El patrón de falla es el mismo: un tablón con margen nulo queda demasiado atrás por tener una prioridad baja y, por tanto, un cociente $\frac{tr}{p}$ alto.

Casos adicionales de la batería manual. Para complementar el análisis, se seleccionaron cuatro casos ilustrativos de la batería incluida en el repositorio. La idea de esta muestra no es agotar todos los tests, sino exhibir rápidamente que el comportamiento del voraz no es uniforme: en algunos casos coincide con el óptimo y en otros se desvía de él por un margen apreciable. La comparación entre costo voraz y costo óptimo para esos cuatro ejemplos se resume en la tabla siguiente.

Test	n	Costo óptimo	Costo voraz	¿Voraz = óptimo?
1	5	44	59	No
5	6	193	193	Sí
12	10	526	638	No
16	20	3758	3816	No

Tabla 5: Casos adicionales tomados de la batería del proyecto.

Si en lugar de mirar solo esos cuatro ejemplos se considera el conjunto completo de los 18 casos del repositorio que sí cuentan con entrada y salida de referencia, el panorama general queda más claro. En esa colección total, el voraz coincidió exactamente con el óptimo en solo 1 caso, es decir, 5,56 %. La diferencia absoluta promedio fue de 32.56 unidades de costo y el exceso relativo promedio alcanzó 16,23 %. El caso más desfavorable dentro de esa batería fue el `test12`, donde el voraz obtuvo 638 frente a un óptimo de 526.

En todo este análisis, si se evalúan m fincas, para cada finca k se calculan dos cantidades:

$$d_k = |C_V^{(k)} - C^{\star(k)}| \quad \text{y} \quad e_k = 100 \cdot \frac{d_k}{C^{\star(k)}}.$$

Aquí d_k es la diferencia absoluta de costo y e_k es esa misma diferencia, pero expresada en porcentaje respecto del óptimo. La división por $C^{\star(k)}$ es importante porque normaliza el error según la escala real de cada finca: una desviación de 10 unidades no tiene el mismo peso si el costo óptimo era 100 que si era 5000.

Con esa notación:

$$\text{diferencia promedio} = \frac{1}{m} \sum_{k=1}^m d_k, \quad \% \text{ diferencia promedio} = \frac{1}{m} \sum_{k=1}^m e_k.$$

En este informe, **% diferencia promedio** y **exceso relativo promedio** significan lo mismo.

Por ejemplo, si en una finca el voraz obtiene costo 110 y el óptimo es 100, entonces

$$d_k = 10 \quad \text{y} \quad e_k = 10 \%.$$

Si en otra finca el voraz también se desvía en 10 unidades, pero el óptimo vale 1000, entonces el error relativo baja a 1 %. Este contraste muestra por qué no basta con la diferencia absoluta: dos errores iguales en costo pueden tener importancias muy distintas dentro del problema.

En palabras simples, la **diferencia promedio** responde a la pregunta “*cuántas unidades de costo se equivoca el voraz, en promedio?*”. En cambio, la **% diferencia promedio** responde a “*qué tan grave es esa equivocación respecto del mejor resultado posible?*”. Por eso conviene reportar ambas: la primera muestra el tamaño bruto del error y la segunda permite juzgar si ese error es pequeño o grande para la finca concreta.

Validación empírica con 5000 casos aleatorios de $n = 20$. Además de la batería manual, se analizó el bloque `n20` definido por la versión actual del script `generar_tests_aleatorios.py`. En su estado actual, el generador produce únicamente instancias con $n = 20$, con un total de 5000 fincas y semilla fija 20260513. Cada tablón se construye a partir de un perfil aleatorio:

Perfil	Regla de generación	Intención del perfil
Crítico	$ts \in [5, 15], slack = 0, p \in [1, 4]$	Fuerza casos borde donde el tablón no admite espera.
Urgente	$ts \in [5, 10], slack \in [1, 3], p \in [3, 4]$	Modela tablonos con poca holgura y alta prioridad.
Balanceado	$ts \in [7, 13], slack \in [2, 6], p \in [1, 4]$	Representa el comportamiento intermedio más frecuente.
Relajado	$ts \in [10, 15], slack \in [7, 13], p \in [1, 4]$	Introduce holgura amplia, permitiendo “falsos urgentes” al no restringir la prioridad.
Largo plazo	$ts \in [20, 100], slack \in [10, 50], p \in [1, 4]$	Añade tablonos que rompen la homogeneidad temporal, dificultando la visión miope del algoritmo.

Tabla 6: Reglas de construcción del generador aleatorio actualizado.

Variable de control. Aquí se usa explícitamente la variable

$$slack = ts - tr,$$

porque esa magnitud controla la urgencia real del tablón. En todos los perfiles se cumple además que $tr = ts - slack$ y $rp \in [0, slack]$.

Distribución nominal del generador. Cada tablón se genera de forma independiente con las siguientes probabilidades:

- 10 % crítico;
- 30 % urgente;
- 25 % balanceado;
- 20 % relajado;
- 15 % largo plazo.

Por qué el generador es válido. La generación siempre preserva las restricciones del enunciado:

$$ts \geq tr, \quad 1 \leq p \leq 4, \quad 0 \leq rp \leq ts - tr.$$

Además, por construcción:

- se cubren simultáneamente casos borde ($ts = tr$), tablonos urgentes, balanceados y relajados;

- no se copian literalmente los casos manuales del proyecto;
- se obtiene una familia sintética coherente con el enunciado y con el estilo de datos observado en la batería real.

Por qué se fijó $n = 20$. Esta decisión tiene dos ventajas metodológicas:

- aísla el comportamiento del criterio voraz en un tamaño ya exigente, sin mezclarlo con el efecto del crecimiento de n ;
- todavía permite resolver todas las instancias por programación dinámica, de modo que cada salida de `roV` se compara contra un óptimo exacto.

Tamaño y validez de la muestra. El bloque usa 5000 fincas de $n = 20$, es decir, 100000 tablonos. Esto hace que el experimento no dependa de unas pocas observaciones afortunadas y, al mismo tiempo, permite comparar cada salida de `roV` contra un óptimo exacto calculado por programación dinámica.

- Bajo la distribución nominal, cada finca contiene en promedio 2.6 tablonos críticos, 7 urgentes, 6 balanceados y 4.4 relajados.
- La probabilidad teórica de que una finca contenga al menos un tablón de cada tipo es muy alta: 93,83 % para críticos, 99,98 % para urgentes, 99,92 % para balanceados y 99,31 % para relajados.
- En el bloque real `n20`, las frecuencias observadas quedaron prácticamente pegadas a las nominales: 13,019 % críticos, 34,971 % urgentes, 29,959 % balanceados y 22,051 % relajados.
- Además, el 94,30 % de las fincas contiene al menos un tablón con holgura cero, más del 99,8 % contiene al menos un urgente o balanceado, y el 100 % contiene al menos un tablón con holgura menor o igual a 3 y uno de prioridad 3 o 4.

En conjunto, esto muestra que la muestra no está sesgada hacia instancias “fáciles” y que el voraz se enfrenta a conflictos reales entre prioridad, holgura, tiempo de riego y día perfecto.

Conclusión metodológica. En síntesis, el experimento es empíricamente sólido porque el generador respeta las restricciones formales, cubre perfiles distintos en proporciones controladas e introduce trampas heurísticas relevantes; además, usa semilla fija y compara siempre contra el costo óptimo real. Bajo esas condiciones, los resultados no dependen de coincidencias fortuitas.

El resultado del criterio actual sobre ese bloque fue:

Estos resultados muestran una idea central: el voraz coincide pocas veces con el orden óptimo exacto (4,38 %), pero el daño económico promedio que produce sigue siendo pequeño (1,97 %). Es decir, falla con frecuencia en la permutación exacta, pero normalmente no se aleja mucho del costo óptimo.

Métrica	Resultado en 5000 casos con $n = 20$
Coincidencias exactas con el óptimo	$219/5000 = 4,38 \%$
Diferencia absoluta promedio	66.94
Exceso relativo promedio frente al óptimo	1,97 %
Peor diferencia absoluta observada	382

Tabla 7: Resumen empírico del criterio voraz actual sobre el bloque oficial actualizado n20.

Para entender de dónde sale esa diferencia, se realizó un **experimento de aislamiento** por perfil. En lugar de mezclar tipos de tablonos, se generaron bloques de 5000 fincas compuestas al 100 % por un solo perfil: crítico, urgente, balanceado, largo plazo o relajado, manteniendo la misma semilla fija 20260513 usada por el generador oficial. La tabla siguiente muestra cómo cambia el rendimiento del voraz en cada uno de esos escenarios puros:

Perfil exclusivo en la Finca ($n = 20$)	Diferencia promedio	Exceso relativo %	Tasa de coincidencia exacta
Fincas 100 % Críticos	0.00	0.00 %	5000/5000 (100 %)
Fincas 100 % Urgentes	2.83	0.05 %	1735/5000 ($\approx 35 \%$)
Fincas 100 % Balanceados	11.13	0.35 %	533/5000 ($\approx 11 \%$)
Fincas 100 % Largo plazo	180.25	1.41 %	25/5000 ($\approx 0.5 \%$)
Fincas 100 % Relajados	62.37	5.53 %	4/5000 ($\approx 0.1 \%$)

Tabla 8: Rendimiento del experimento paralelo aislando la heurística contra 5000 fincas de exclusividad pura por perfil.

Estas métricas explican bien lo ocurrido en el bloque real de 5000 casos (219/5000). En perfiles rígidos, el voraz funciona muy bien: es perfecto en el caso 100 % Crítico y en el caso 100 % Urgente mantiene un exceso relativo de apenas 0,05 %, con 1735 coincidencias exactas sobre 5000.

A medida que aumenta la holgura, la coincidencia exacta cae con fuerza: baja a 533/5000 en 100 % Balanceado, a 25/5000 en 100 % Largo plazo y a 4/5000 en 100 % Relajado. Sin embargo, incluso cuando deja de acertar en la permutación exacta, el costo adicional sigue siendo moderado en balanceado y largo plazo, y solo se vuelve claramente más visible en relajado. En resumen, el voraz falla sobre todo al ordenar tareas muy posponibles, pero se mantiene fuerte donde retrasarse realmente cuesta caro.

Sustento empírico del criterio elegido. Para justificar inequívocamente la selección de $\frac{tr}{p}$ frente a intuiciones alternativas que a priori podrían parecer razonables (como atender lo más urgente, de mayor prioridad o el día perfecto), se ejecutó un laboratorio comparando distintas combinaciones implementadas en `backend/src/voraz_criterios.cpp`. El ranking sobre el nuevo conjunto mixto de 5000 instancias arrojó los siguientes resultados:

Los datos dejan una conclusión simple: cuando $\frac{tr}{p}$ deja de ser el criterio principal, el desempeño empeora de forma marcada. En cambio, cuando se mantiene como criterio

Criterio principal + desempate	% diferencia promedio	Diferencia promedio	Coincidencias exactas
$\frac{tr}{p} + ts - tr$	1.97 %	66.94	219/5000
$\frac{tr}{p} + \text{supervivencia}$	1.99 %	67.80	201/5000
$\frac{tr}{p} + \text{prioridad alta}$	2.15 %	71.30	165/5000
Supervivencia + $\frac{tr}{p}$	$\approx 14,8 \%$	$> 700,0$	0/5000
Prioridad alta + $\frac{tr}{p}$	$\approx 8,4 \%$	$> 400,0$	0/5000

Tabla 9: Comparativo manteniendo $\frac{tr}{p}$ como criterio principal frente a priorizar otros atributos.

base y se desempata con la holgura $ts - tr$, se obtienen los mejores resultados. Por eso, dentro de las heurísticas rápidas evaluadas, esta combinación resulta la más robusta para el problema.

4.3. Complejidad y Corrección

Complejidad. El algoritmo voraz ejecuta dos fases:

1. ordena los n tableros según el criterio $\frac{tr}{p}$ con desempate por holgura;
2. evalúa la permutación resultante.

La primera fase domina el costo total, por lo que la complejidad temporal es

$$O(n \log n).$$

La simulación posterior solo agrega un recorrido lineal $O(n)$. El espacio auxiliar también es $O(n)$, debido al arreglo de índices.

Correctitud del algoritmo voraz. El algoritmo voraz no siempre da la respuesta correcta al problema, entendiendo por respuesta correcta una programación de riego con costo óptimo. Aunque el problema sí tiene subestructura óptima, el criterio local usado por el algoritmo no garantiza la propiedad de escogencia voraz. Por esta razón, escoger en cada paso el tablero que parece más conveniente según $\frac{tr}{p}$ no asegura necesariamente obtener la mejor solución global.

En consecuencia, el algoritmo debe entenderse como una heurística eficiente: puede coincidir con el óptimo en ciertos tipos de instancias, especialmente cuando el retraso domina el costo, pero no es exacto para el caso general. A continuación se justifica esta afirmación mediante la subestructura óptima del problema, un contraejemplo a la escogencia voraz y el análisis de los casos en los que el criterio tiende a acertar o fallar.

Subestructura óptima. Sí se cumple. Sea

$$\Pi^* = \langle \pi_0^*, \pi_1^*, \dots, \pi_{n-1}^* \rangle$$

una programación óptima. Si fijamos el primer tablón π_0^* , entonces el resto de la programación siempre comienza después de $tr_{\pi_0^*}$. Por tanto, el sufijo

$$\langle \pi_1^*, \dots, \pi_{n-1}^* \rangle$$

debe ser óptimo para ese subproblema residual. Si no lo fuera, existiría otro orden sobre esos mismos tableros con menor costo; al reemplazar solo el sufijo en Π^* , se obtendría una programación total más barata, contradicción.

Por ejemplo, si una solución óptima fuera $\langle 2, 0, 1, 3 \rangle$, entonces el sufijo $\langle 0, 1, 3 \rangle$ debe ser el mejor posible una vez regado T_2 . Si existiera un sufijo mejor, como $\langle 1, 0, 3 \rangle$, entonces $\langle 2, 1, 0, 3 \rangle$ costaría menos que la supuesta solución óptima. Por lo tanto, el problema sí tiene subestructura óptima.

Propiedad de escogencia voraz. No se cumple en general. Considérese la finca con dos tableros

$$T_0 = \langle 10, 2, 1, 0 \rangle, \quad T_1 = \langle 5, 3, 4, 0 \rangle.$$

Como

$$\frac{tr_0}{p_0} = 2,00, \quad \frac{tr_1}{p_1} = 0,75,$$

el criterio voraz elige primero a T_1 . Sin embargo:

Permutación	Cálculo	Costo total
$\langle 1, 0 \rangle$	$T_1 : 5 - (0 + 3) = 2, T_0 : 2(10 - (3 + 2)) = 10$	12
$\langle 0, 1 \rangle$	$T_0 : 10 - (0 + 2) = 8, T_1 : 2(5 - (2 + 3)) = 0$	8

Tabla 10: Contraejemplo a la propiedad de escogencia voraz.

La solución óptima comienza por T_0 , no por el tablón de menor $\frac{tr}{p}$. Esto demuestra formalmente que no existe garantía de que una solución óptima empiece con la escogencia voraz.

Lectura conjunta de la demostración formal y la evidencia experimental. El algoritmo voraz no es exacto para el problema general porque el costo combina tres regímenes distintos. En particular, los casos 1 y 2 premian en ocasiones esperar, mientras que el caso 3 castiga fuertemente el retraso. Esa mezcla rompe la propiedad de escogencia voraz. Sin embargo, el argumento de intercambio presentado en la Sección 4.1 sigue siendo válido en el régimen tardío y explica por qué $\frac{tr}{p}$ funciona razonablemente bien en la práctica.

Cuándo tiende a acertar y cuándo falla. Con base en la teoría y en los experimentos del proyecto, el criterio tiende a acertar mejor en perfiles donde retrasarse es peligroso y la urgencia temporal domina la decisión:

- **Crítico:** aquí el voraz acierta casi siempre, porque los tableros no tienen holgura ($ts - tr = 0$) y postergar cualquiera de ellos es inmediatamente costoso; en ese entorno, ordenar por $\frac{tr}{p}$ refleja muy bien la presión real del problema.

- **Urgente:** también suele comportarse bien, porque la holgura es pequeña y los tablonos están cerca del régimen tardío; por eso las decisiones locales del criterio se parecen bastante a las decisiones del óptimo.
- **Instancias con tablonos cortos y de prioridad alta:** el cociente $\frac{tr}{p}$ favorece precisamente esos tablonos, de modo que el algoritmo protege primero tareas que son rápidas de regar y costosas de retrasar.
- **Casos favorables al día perfecto:** a veces el orden voraz coincide accidentalmente con un buen rp ; cuando eso ocurre, el costo mejora aun sin que el criterio haya usado rp de forma explícita.

Por el contrario, el criterio falla más cuando predominan perfiles con mucha holgura, porque allí no siempre conviene regar primero lo que parece más rentable según $\frac{tr}{p}$:

- **Relajado:** estos tablonos tienen holgura amplia, así que el algoritmo pierde precisión al decidir cuál conviene posponer; por eso aquí baja mucho la coincidencia exacta con el óptimo.
- **Largo plazo:** este perfil rompe la homogeneidad temporal del problema; como sus tablonos pueden esperar bastante, el criterio local deja de capturar bien el orden global más conveniente.
- **Balanceado:** se ubica en una zona intermedia; no es tan adverso como relajado o largo plazo, pero ya no es tan rígido como crítico o urgente, por lo que el voraz acierta menos.
- **Instancias donde rp es decisivo:** el criterio principal no incorpora el día perfecto, de modo que puede producir un orden económicamente razonable pero no el orden exacto que minimiza el costo total.
- **Mezclas de perfiles frágiles con perfiles holgados:** una decisión local que parece eficiente sobre un tablón posponible puede desplazar después a otro más sensible y generar errores en cadena.

En consecuencia:

- el criterio $\frac{tr}{p}$ tiene una justificación formal sólida en el subproblema tardío;
- el problema completo sí tiene subestructura óptima, pero no satisface la propiedad de escogencia voraz;
- por tanto, el algoritmo es una heurística eficiente y bien argumentada, pero no un método exacto para el caso general.

5. Solución Mediante Programación Dinámica

5.1. Subestructura Óptima

La solución dinámica del proyecto modela cada subproblema como un subconjunto de tableros ya regados. Sea $S \subseteq \{0, \dots, n-1\}$ un subconjunto. Definimos:

$$\tau(S) = \sum_{i \in S} tr_i,$$

que representa el tiempo total consumido por los tableros de S . Esta cantidad depende solo del subconjunto y no del orden interno.

Sea $OPT(S)$ el costo mínimo de regar exactamente los tableros de S en los primeros $|S|$ lugares de la programación. Si una programación óptima para S termina con el tablero $i \in S$, entonces el prefijo formado por $S \setminus \{i\}$ debe ser una solución óptima para ese subconjunto. Si no lo fuera, podría reemplazarse por una de menor costo y se obtendría una mejor programación para S , contradicción.

Esa observación demuestra la propiedad de subestructura óptima. Además, el número total de subproblemas es 2^n , uno por cada subconjunto de tableros.

5.2. Definición Recursiva

Para cada tablero i , definimos la función de costo local en función del instante de inicio t :

$$c_i(t) = \begin{cases} ts_i - (t + tr_i), & \text{si } t = rp_i, \\ 2(ts_i - (t + tr_i)), & \text{si } t \leq ts_i - tr_i \text{ y } t \neq rp_i, \\ 2p_i((t + tr_i) - ts_i), & \text{en otro caso.} \end{cases}$$

La recurrencia natural, agregando el último tablero del subconjunto, es:

$$OPT(\emptyset) = 0,$$

$$OPT(S) = \min_{i \in S} \{OPT(S \setminus \{i\}) + c_i(\tau(S) - tr_i)\}.$$

La implementación del proyecto usa la misma idea en sentido hacia adelante. Si un estado está representado por una máscara de bits M , y el tiempo consumido hasta ese estado es $\tau(M)$, entonces para cada tablero $i \notin M$ se intenta la transición:

$$DP[M \cup \{i\}] = \min(DP[M \cup \{i\}], DP[M] + c_i(\tau(M))).$$

Además del arreglo de costos mínimos, la implementación almacena el estado anterior y el último tablero agregado, lo cual permite reconstruir una permutación óptima al finalizar.

5.3. Análisis de Complejidad Temporal y Espacial

La programación dinámica recorre los 2^n estados y, desde cada uno, intenta agregar cada tablón aún no regado. Por ello, el número total de transiciones es del orden de $n2^n$, y la complejidad temporal es:

$$O(n2^n).$$

El espacio está dominado por los arreglos indexados por máscara, cada uno de tamaño 2^n . Por tanto, la complejidad espacial es:

$$O(2^n).$$

En la implementación concreta se usan varias tablas auxiliares para costo, padre, último tablón y tiempo acumulado. Eso aumenta el factor constante, pero no cambia el orden asintótico.

Si se analiza la utilidad práctica bajo la hipótesis del enunciado de 3×10^8 operaciones por minuto y 4 bytes por celda, se obtiene el siguiente panorama aproximado para $n = 2^k$:

k	$n = 2^k$	$n2^n$ operaciones	Tiempo aproximado	Memoria base $4 \cdot 2^n$
3	8	2,048	despreciable	1 KB
4	16	1,048,576	0.0035 min	256 KB
5	32	137,438,953,472	458 min	16 GB

Tabla 11: Estimación práctica de la programación dinámica.

La conclusión es que la programación dinámica es perfectamente viable para tamaños pequeños y medianos, pero deja de ser práctica cuando $k \geq 5$, es decir, cuando n ya alcanza 32 tablones. En ese punto, tanto el tiempo como la memoria crecen demasiado rápido. Aun así, sigue siendo una mejora enorme frente a la fuerza bruta, cuyo crecimiento factorial la vuelve inviable mucho antes.

6. Comparación de Estrategias

La Tabla 12 resume las principales diferencias entre los tres enfoques desarrollados.

Desde el punto de vista de calidad de solución, fuerza bruta y programación dinámica son los enfoques correctos para el problema general. La diferencia entre ellos está en la eficiencia: la programación dinámica evita recalculer subproblemas equivalentes y por eso reemplaza el crecimiento factorial por uno exponencial de base 2.

Desde el punto de vista de tiempo de respuesta, el voraz es claramente superior. Su costo $O(n \log n)$ lo hace apropiado para interfaces interactivas o escenarios en los que se necesite una estimación rápida. No obstante, los resultados observados en la batería del proyecto muestran que esta ganancia en velocidad tiene un precio importante en calidad: el voraz no captura adecuadamente la interacción entre prioridad, margen de supervivencia y día perfecto.

En consecuencia, una arquitectura razonable para el proyecto es usar:

Estrategia	Complejidad temporal	Garantía de optimalidad	Uso práctico
Fuerza bruta	$O(n!n)$	Sí	Solo instancias muy pequeñas; sirve como referencia exacta.
Voraz	$O(n \log n)$	No	Muy rápido; útil para obtener una respuesta inmediata.
Programación dinámica	$O(n2^n)$	Sí	Exacta y mucho más escalable que fuerza bruta para instancias moderadas.

Tabla 12: Comparación general de las estrategias.

- fuerza bruta como validador para instancias pequeñas;
- programación dinámica como método exacto principal;
- voraz como alternativa rápida cuando el tamaño de entrada o el contexto de uso desaconsejan un algoritmo exacto.

7. Conclusiones

El problema del riego óptimo no puede resolverse correctamente con una intuición local simple, porque el costo de cada tablón depende del instante exacto en que comienza su riego y ese instante, a su vez, depende de todas las decisiones previas. Esa dependencia global explica por qué el enfoque voraz, aunque eficiente, no siempre logra soluciones óptimas.

La fuerza bruta ofrece una solución conceptualmente sencilla y totalmente correcta, pero su complejidad factorial la limita a instancias muy pequeñas. La programación dinámica, en cambio, aprovecha la subestructura óptima del problema y permite obtener soluciones exactas con una mejora sustancial en eficiencia. Aun así, su crecimiento exponencial impone un límite práctico para valores grandes de n .

En este proyecto, la mejor relación entre exactitud y escalabilidad la ofrece la programación dinámica. El algoritmo voraz resulta valioso como heurística y como punto de comparación experimental, pero no reemplaza un método exacto cuando se requiere garantizar optimalidad. La comparación entre los tres enfoques muestra precisamente la idea central del curso: distintas estrategias de diseño algorítmico pueden modelar el mismo problema, pero sus beneficios y limitaciones son profundamente diferentes.